Corporate Project Management System

Complete Project Documentation

Full-Stack Web Application with AI/ML Intelligence

Project	Corporate Project Management System
Author	Shivani Chaudhari
Course	Data Storage Technologies — Final Project
Date	April 2026
Version	2.0 — Updated with completed AI/ML features

1. Project Overview

The Corporate Project Management System is a full-stack enterprise web application designed to centralize and intelligently manage corporate projects, tasks, documents, and team members. The system moves beyond traditional project management tools by embedding an AI/ML intelligence layer that predicts problems, automates decisions, and surfaces insights — enabling teams to shift from reactive firefighting to proactive, data-driven project management.

One-Line Pitch

A full-stack corporate project management platform with an embedded AI/ML intelligence layer that automatically classifies documents, prioritizes tasks by risk, enables semantic search, predicts project delays, and sends proactive alerts — deployed on Microsoft Azure with a CI/CD pipeline.

1.1 Problem Statement

In large organizations, employees waste significant time searching for documents, manually prioritizing tasks, and reacting to missed deadlines rather than preventing them. Without a centralized system:

- Employees store files inconsistently across drives, email, and cloud storage
- Important documents get misplaced or exist in multiple conflicting versions
- Deadlines are missed because there is no early warning system
- Project managers lack a unified view of workload and progress
- Time is wasted locating relevant information across disconnected tools

1.2 Solution

The system implements a relational database solution with a structured API layer, a modern React frontend, and a Python AI microservice. The AI layer learns from the data being created — documents uploaded, tasks managed, projects tracked — and uses this to make intelligent predictions and automate routine decisions.

1.3 Key Features Implemented

- Authentication — JWT-based login and registration with role-based access (Admin, Manager, Staff)
- Project Management — Full CRUD for projects with status tracking and team assignment
- Task Management — Task creation, priority setting, due date tracking, overdue detection
- Document Management — Document metadata storage with category classification and version tracking
- Employee Management — Team member profiles with project assignment history
- Dashboard Analytics — Real-time KPI cards, bar charts, project status distribution
- AI/ML Layer — Features 1, 2, 3, and 5 implemented; Feature 4 (Task Auto-Prioritization) planned

2. Technology Stack

The system is built on a three-service architecture — a React frontend, a Node.js backend, and a Python AI microservice — each deployed independently and communicating via REST APIs.

2.1 Architecture Overview

Layer	Technology	Purpose
Frontend	React 18 + TypeScript + Vite	User interface and interaction
Styling	Tailwind CSS v4	Utility-first responsive styling
State Mgmt	TanStack React Query	Server state, caching, refetching
Routing	React Router v6	Client-side page navigation
HTTP Client	Axios	API requests with interceptors
Charts	Recharts	Dashboard data visualizations
Backend	Node.js + Express.js	REST API server
ORM	Prisma v6	Database access and schema management
Auth	JWT + bcryptjs	Secure authentication
Scheduler	node-cron	Nightly alert jobs
AI Service	Python 3.11 + FastAPI	ML model serving
ML Library	scikit-learn	Classification and regression models
NLP	sentence-transformers	Semantic vector embeddings
Vector Search	FAISS	Similarity search index
Database	Azure SQL (Prisma v6)	Relational data storage
File Storage	Azure Blob Storage	Document file storage
Hosting	Azure App Service	Cloud deployment
CI/CD	GitHub Actions	Automated build and deploy
Pkg Manager	pnpm	Fast JavaScript package management
AI Env	Anaconda (conda)	Python environment management

2.2 Frontend Technologies

React 18 + TypeScript

React is a JavaScript library by Meta for building user interfaces using reusable components. TypeScript adds type safety — catching bugs at compile time before they reach production. Every component, API call, and data model in the frontend is fully typed.

Tailwind CSS v4

Tailwind is a utility-first CSS framework. Version 4 removed the old config file and init command — setup is done entirely through a Vite plugin. The entire UI uses a custom dark theme with CSS variables defined in globals.css.

TanStack React Query

React Query manages all server state — loading states, caching, background refetching, and error handling — automatically, eliminating manual fetch/loading/error logic.

2.3 Backend Technologies

Node.js + Express.js

The backend exposes 22+ REST API endpoints across 6 resource types: auth, employees, projects, tasks, documents, and categories.

Prisma ORM v6

Prisma is an Object-Relational Mapper that lets you write database queries in TypeScript instead of raw SQL. Version 6 was used instead of v7 due to stability issues with the v7 SQL Server adapter.

JWT Authentication

JSON Web Tokens provide secure, stateless authentication. After login the server issues a signed token stored in localStorage and sent with every request via the Authorization header.

node-cron

A task scheduler for Node.js used for the nightly overdue task check — runs every night at midnight, queries all incomplete tasks past their due date, and triggers the AI alert pipeline.

2.4 AI Service Technologies

Python FastAPI

FastAPI is a modern Python web framework for building APIs. The AI service runs as a separate microservice on port 8000 and is called by the Node.js backend.

scikit-learn

Used for the document classification model (Multinomial Naive Bayes) and the project delay prediction model (Logistic Regression). Lightweight and does not require a GPU.

sentence-transformers

Provides pre-trained models for converting text into semantic vector embeddings. The all-MiniLM-L6-v2 model (trained on 1 billion sentence pairs) powers the NLP semantic search feature.

FAISS

Facebook AI Similarity Search — used as the search index for the NLP feature. All document titles and task names are stored as vectors, and user queries are matched using cosine similarity.

2.5 Azure Cloud Services

Azure SQL Database (Basic S0 Tier)

Hosts the production database with all 7 tables. The Basic S0 tier costs approximately $5/month — within the $100 Azure for Students credit.

Azure Blob Storage

Object storage hosting uploaded project documents. The 'documents' container is set to private access, requiring authentication to retrieve files.

Azure App Service (F1 Free Tier)

Hosts all three services (frontend, backend, AI service) in the cloud. The F1 free tier costs $0/month.

3. Database Design

The database follows a normalized relational structure hosted on Azure SQL and managed through Prisma ORM.

3.1 Tables

Table	Primary Key	Purpose	Key Fields
Employee	EmployeeID	Stores team member profiles	FirstName, LastName, Email, Password, Role
Project	ProjectID	Stores project records	ProjectName, ClientName, StartDate, EndDate, Status
Task	TaskID	Tracks work items	TaskName, DueDate, Status, Priority, ProjectID (FK)
Document	DocumentID	Document metadata	DocumentTitle, FileName, VersionNumber, ProjectID (FK), CategoryID (FK)
Category	CategoryID	Document classification groups	CategoryName, Description
FileLocation	FileLocationID	Physical file storage paths	FilePath, LocationType, Notes
ProjectAssignment	AssignmentID	Employee-Project many-to-many	EmployeeID (FK), ProjectID (FK), RoleInProject

3.2 Key Relationships

- One-to-Many: Project → Tasks (one project has many tasks)
- One-to-Many: Project → Documents (one project has many documents)
- Many-to-One: Document → Category (many documents belong to one category)
- Many-to-One: Document → FileLocation (many documents stored at one location)
- Many-to-Many: Employee ↔ Project (resolved via ProjectAssignment junction table)

3.3 Indexing Strategy

Index	Column(s)	Purpose
idx_project_status	Project(Status)	Faster project status filtering
idx_document_project	Document(ProjectID)	Faster document retrieval by project
idx_task_project	Task(ProjectID)	Faster task lookup for workload dashboards
idx_assignment_emp_proj	ProjectAssignment(EmployeeID, ProjectID)	Faster employee-project lookups

3.4 Migration History

- init — Created all 7 base tables with foreign key constraints

- add_password_v6 — Added Password field to Employee table for authentication

4. Backend API Reference

The backend exposes a RESTful API with 22 endpoints across 6 resource groups. All endpoints except auth require a valid JWT token in the Authorization header.

4.1 Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register a new employee account
POST	/api/auth/login	Login and receive JWT token

4.2 Projects

Method	Endpoint	Description
GET	/api/projects	Get all projects with task/doc counts and team
GET	/api/projects/:id	Get single project with full details
POST	/api/projects	Create a new project
PUT	/api/projects/:id	Update project details or status
DELETE	/api/projects/:id	Delete a project

4.3 Tasks

Method	Endpoint	Description
GET	/api/tasks	Get all tasks (filterable by projectId, status)
GET	/api/tasks/overdue	Get all tasks past due date and not completed
POST	/api/tasks	Create a new task
PUT	/api/tasks/:id	Update task details, status, or priority
DELETE	/api/tasks/:id	Delete a task

4.4 Documents, Employees, Categories

Method	Endpoint	Description
GET	/api/documents	Get all documents (filterable by projectId, categoryId)
POST	/api/documents	Add document metadata record
PUT	/api/documents/:id	Update document title, category, version
DELETE	/api/documents/:id	Delete document record
GET	/api/employees	Get all employees
GET	/api/employees/:id	Get employee with assignment history
PUT	/api/employees/:id	Update employee details

GET	/api/categories	Get all categories with document counts
POST	/api/categories	Create a new category
DELETE	/api/categories/:id	Delete a category

5. Frontend Application

The frontend is a single-page React application with a dark theme UI (deep navy #0a0f1e background, electric cyan #00d4ff primary accent, violet #7c3aed secondary accent). Typography uses Syne for headings and DM Sans for body text.

5.1 Pages Built

Page	Route	Key Features
Login	/login	Email/password form, JWT storage, animated background grid
Dashboard	/dashboard	KPI metric cards, bar chart (Recharts), project status panel, AI insight block, recent tasks and projects
Projects	/projects	Filter tabs by status, project cards with stats (tasks/docs/members), create/edit/delete modal
Tasks	/tasks	Table layout, dual filters (status + priority), overdue row highlighting, create/edit/delete modal
Documents	/documents	Card grid with file type icons, category filter tabs, version badges, metadata display
Employees	/employees	Profile cards with gradient avatars, role badges, project assignment count
Categories	/categories	Color-coded cards with document count, inline create form

6. AI/ML Features

Implementation Status: Features 1 (Document Classification), 2 (Semantic Search), 3 (Delay Prediction), and 5 (Predictive Alerts) are fully implemented. Feature 4 (Task Auto-Prioritization) is planned for a future sprint.

6.1 Architecture

The AI service is a separate Python FastAPI microservice running on port 8000. The Node.js backend calls it internally — the frontend never communicates with the AI service directly. This separation keeps ML dependencies isolated and allows the AI service to be scaled independently.

Data Flow:

Frontend → Backend (Node.js :5000) → AI Service (FastAPI :8000) → ML Model → Response back through the chain

6.2 Feature Overview

#	Feature	ML Type	Algorithm	Feeds Into
1	Document Auto-Classification	Supervised — Multi-class	Multinomial Naive Bayes	Auto-fills category on document upload
2	NLP Semantic Search	Unsupervised — Similarity search	Sentence Transformers + FAISS	Global search bar in topbar
3	Project Delay Prediction	Supervised — Binary classification	Logistic Regression / Random Forest	Drives Features 4 and 5
4	Task Auto-Prioritization	Rule-based + ML score	Weighted scoring + delay score	Updates task priority automatically
5	Predictive Alerts	Threshold classification	Delay score + urgency rules	Proactive notifications before deadline

6.3 Feature Details

Feature 1 — Auto Document Classification ✓ Complete

Problem: Employees upload documents without categorizing them properly, making retrieval slow and inconsistent.

Solution: When a document is uploaded, its title is sent to the AI service which predicts the most likely category (Technical, Finance, Report, HR) with a confidence score. The category auto-fills in the upload form.

Algorithm: Multinomial Naive Bayes trained on (DocumentTitle, CategoryName) pairs. A TF-IDF vectorizer with bigrams (1,2 n-gram range) converts titles to feature vectors. The model is trained lazily on first call and cached in memory.

Endpoint: POST /classify-document

```
Input:  { "title": "System Architecture Design" }
Output: { "category": "Technical", "confidence": 0.89 }
```

Feature 2 — NLP Semantic Search ✓ Complete

Problem: Standard keyword search fails when users do not remember exact file names. Searching 'login security' finds nothing even though 'Authentication Module Design' exists.

Solution: All document titles and task names are converted to vector embeddings on startup and stored in a FAISS index. User queries are also embedded and matched by cosine similarity — returning semantically relevant results even when exact words differ.

Algorithm: Sentence Transformers (all-MiniLM-L6-v2, trained on 1 billion sentence pairs) + FAISS IndexFlatIP for inner-product (cosine) similarity. Embeddings are normalized before indexing to ensure cosine distance.

Endpoint: POST /search

```
Input:  { "query": "login security", "limit": 5 }
Output: { "results": [{ "title": "Authentication Module Design", "score": 0.82 }] }
```

Feature 3 — Project Delay Prediction ✓ Complete

Problem: Managers discover projects are delayed only after the deadline has passed, leaving no time to intervene.

Solution: A classification model takes project health metrics as input and predicts delay risk (On Track / At Risk / Likely Delayed) with a score 0–100. Dashboard shows a risk badge on each project card.

Algorithm: Logistic Regression trained on 6,000 synthetic records generated from realistic project parameters. Features: total tasks, completed tasks, overdue tasks, days until deadline, team size. A StandardScaler normalizes inputs. Risk score is a probability-weighted sum (weights: On Track=18, At Risk=52, Likely Delayed=86).

Endpoint: POST /predict-delay

```
Input:  { "totalTasks": 20, "completedTasks": 8, "overdueTasks": 3, "daysUntilDeadline": 15,
"teamSize": 3 }
Output: { "riskScore": 78, "riskLevel": "Likely Delayed", "reason": "3 task(s) overdue;
completion rate is behind expected pace." }
```

Feature 4 — Task Auto-Prioritization ✗ Planned

Problem: Task priority is set manually by users and never updated, even when project conditions change drastically.

Solution (Planned): When the delay prediction model flags a project as At Risk or Likely Delayed, all incomplete tasks in that project will be automatically re-scored. Tasks close to their deadline in at-risk projects get escalated. The UI will show an Auto badge on escalated tasks.

Algorithm (Planned): Rule-based scoring formula combining days until due (40%), project delay score (35%), and original priority label (25%). Tasks scoring above threshold get escalated.

Endpoint (Planned): POST /prioritize-tasks

Feature 5 — Predictive Alerts (Combined Pipeline) ✓ Complete

Problem: Alerts fire after tasks are already overdue. Managers find out about problems too late to fix them.

Solution: A nightly node-cron job calls the AI service with all project and task data. The delay prediction model identifies at-risk projects, and the alert classifier generates severity-ranked messages. Additionally, incomplete non-High-priority tasks in at-risk projects are automatically escalated.

Algorithm: Three-stage pipeline — (1) Delay prediction classifies project risk level, (2) Alert builder assigns severity (Critical / Warning / Notice) based on days until deadline and risk level, (3) Escalation engine bumps task priority by one level for qualifying tasks.

Alert Severity Rules: Critical if project is overdue or Likely Delayed with < 14 days left; Warning if At Risk or deadline < 21 days; Notice if On Track.

Endpoint: POST /analyze-project-health

```
Input:  { "projectId": 1, "tasks": [...], "daysUntilDeadline": 5 }
Output: { "riskLevel": "Likely Delayed", "riskScore": 82, "alerts": [{"type":
"ProjectAtRisk", "severity": "Critical"}], "escalatedTasks": [{"taskId": 3, "fromPriority":
"Medium", "toPriority": "High"}] }
```

6.4 Training Data Sources

Feature	Dataset	Source	Size
Document Classification	BBC News Dataset	Kaggle	1,490 training + 736 test records
Project Delay Prediction	SEERA Dataset / NASA PROMISE	Kaggle / promise.site.uottawa.ca	1,000+ software projects
NLP Semantic Search	No external data needed	Pre-trained model (1B sentence pairs)	N/A
Task Auto-Prioritization	No external data needed	Rule-based on own task data	N/A — Planned
Predictive Alerts	No external data needed	Threshold logic on own project data	N/A

6.5 AI File Structure

File	Purpose
`main.py`	FastAPI app entry point, all route definitions
`models/classifier.py`	Document classification — Naive Bayes training and prediction
`models/delay_predictor.py`	Project delay prediction — Logistic Regression / Random Forest
`models/health.py`	Feature 5 combined pipeline — delay model + alert severity + task escalation
`models/search.py`	NLP search — sentence-transformers embedding + FAISS index
`models/prioritizer.py`	Task prioritization — scoring formula and escalation logic (Planned)
`models/alerts.py`	Overdue alert analysis — severity classification
`data/seed_documents.py`	Training data seeding script for document classification
`requirements.txt`	All Python dependencies with pinned versions

7. Setup & Development Environment

7.1 System Requirements

- Windows 10/11
- Node.js LTS v24+ with pnpm package manager
- Python 3.11 via Anaconda distribution
- Git 2.45+
- VS Code with Prisma and ES7+ React Snippets extensions

7.2 Project Structure

Folder	Service	Port
`corporate-pm-system/frontend/`	React + Vite application	5173
`corporate-pm-system/backend/`	Node.js + Express API	5000
`corporate-pm-system/ai-service/`	Python FastAPI ML service	8000
`corporate-pm-system/.github/workflows/`	GitHub Actions CI/CD	N/A

7.3 Running Locally

Three terminals are required simultaneously:

- Backend: cd backend && pnpm dev → runs on http://localhost:5000
- Frontend: cd frontend && pnpm dev → runs on http://localhost:5173
- AI Service: conda activate corporate-pm && uvicorn main:app --reload --port 8000

7.4 Key Challenges Solved

Challenge	Root Cause	Solution Applied
Tailwind v4 init command failed	v4 removed init command and config file	Used @tailwindcss/vite plugin, @import tailwindcss in CSS
pnpm init -y failed	pnpm does not support -y flag unlike npm	Ran pnpm init without flags, pressed Enter for defaults
TypeScript module errors	verbatimModuleSyntax: true in tsconfig.app.json	Set verbatimModuleSyntax and erasableSyntaxOnly to false
Prisma v7 adapter errors	PrismaMssqlAdapterFactory losing config (v7 bug)	Downgraded entire Prisma stack to v6 (stable)
Azure region restriction	East US blocked on Student subscription	Switched all resources to Central India region
Shadow database required	Azure SQL blocks auto shadow DB creation	Created second database project-management-shadow manually
Frontend CORS error	Browser blocks cross-origin requests to localhost:5000	Added Vite proxy /api → localhost:5000 in vite.config.ts

Azure IP firewall blocking	Dynamic ISP IP changes daily	Added AllowAll firewall rule (0.0.0.0 to 255.255.255.255)

8. Azure Cloud Setup

8.1 Resources Created

Resource Name	Type	Tier	Region	Monthly Cost
corporate-pm-rg	Resource Group	N/A	Central India	$0
corporate-pm-server	SQL Server	N/A	Central India	$0
project-management-db	SQL Database (Main)	Free General Purpose	Central India	$0
project-management-shadow	SQL Database (Shadow)	Free General Purpose	Central India	$0
corporatepmstorage	Storage Account	Standard LRS	Central India	<$0.10
documents	Blob Container	Private	Central India	$0

8.2 Budget Management

- Total Azure for Students credit: $100 for 12 months
- Estimated monthly spend: under $1 (Free General Purpose SQL tier)
- Spending alert configured at $20 to prevent unexpected charges
- All App Service deployments use F1 Free tier ($0/month)

9. Project Progress Tracker

Phase	Task	Status
Setup	Node.js, Git, VS Code installation	**Complete**
Setup	React + Tailwind v4 frontend	**Complete**
Setup	Node.js + Express backend	**Complete**
Setup	Python FastAPI AI service	**Complete**
Setup	GitHub repository	**Complete**
Setup	Azure SQL Database + Prisma migration	**Complete**
Setup	Azure Blob Storage	**Complete**
Backend	JWT Authentication API	**Complete**
Backend	Projects CRUD API	**Complete**
Backend	Tasks CRUD API	**Complete**
Backend	Documents CRUD API	**Complete**
Backend	Employees API	**Complete**
Backend	Categories API	**Complete**
Frontend	Login page	**Complete**
Frontend	Dashboard with charts	**Complete**
Frontend	Projects page	**Complete**
Frontend	Tasks page	**Complete**
Frontend	Documents page	**Complete**
Frontend	Employees page	**Complete**
Frontend	Categories page	**Complete**
AI/ML	Document Classification (Naive Bayes) — Feature 1	**Complete**
AI/ML	NLP Semantic Search (Sentence Transformers) — Feature 2	**Complete**
AI/ML	Project Delay Prediction (Logistic Regression) — Feature 3	**Complete**
AI/ML	Task Auto-Prioritization — Feature 4	**Planned**
AI/ML	Predictive Alert Pipeline — Feature 5	**Complete**
DevOps	GitHub Actions CI/CD pipeline	**Planned**
DevOps	Azure App Service deployment	**Planned**

End of Documentation — Corporate Project Management System (v2.0)